Causal Machine Learning for Feature Selection in Breast Cancer Data

Aims of this projects:

- Applying ML models and feature importance methods to identify key predictors in biomedical data
- Improving model interpretability while maintaining predictive performance
- Comparing models with full set of predictors and subset of them
- Exploring causal relevance of features by analyzing model sensitivity and robustness

Part I: Predictive modeling & Feature Selection

The outcome variable is binary, indicating the diagnosis of the tumor (benign vs malignant). The task is to classify observations based on feature measurements extracted from biomedical data. This project focuses on predictive modeling and feature importance as a first step toward identifying potentially relevant variables for future causal analysis.

Model 1: RF

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
import pandas as pd

# data
data = load_breast_cancer()
X = data.data
y = data.target

# split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# model
model = RandomForestClassifier()
model.fit(X_train, y_train)

# feature importance
df = pd.DataFrame({
    "feature": data.feature_names,
    "importance": model.feature_importances_
}).sort_values(by="importance", ascending=False)
```

This step loads a well-known biomedical dataset and trains a random forest classifier to model the relationship between input features and cancer diagnosis. The dataset is split into training and testing subsets to ensure that model performance is evaluated on unseen data.

A random forest model is used due to its ability to capture nonlinear relationships and interactions between features. After training, feature importance scores are extracted to identify the most influential variables in the classification task.

Double-click (or enter) to edit

```
from sklearn.metrics import accuracy_score

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.9473684210526315
```

Double-click (or enter) to edit

This step evaluates the performance of the trained model. Predictions are generated on the test dataset, and accuracy is computed to measure how well the model classifies unseen data.

However, high accuracy alone does not guarantee causal understanding, as the model may rely on strong correlations rather than true causal relationships.

```
df.head(10)
```

	feature	importance
20	worst radius	0.125065
7	mean concave points	0.124425
27	worst concave points	0.113854
22	worst perimeter	0.108937
23	worst area	0.098085
13	area error	0.059158
2	mean perimeter	0.056063
6	mean concavity	0.052347
3	mean area	0.051697
0	mean radius	0.022959

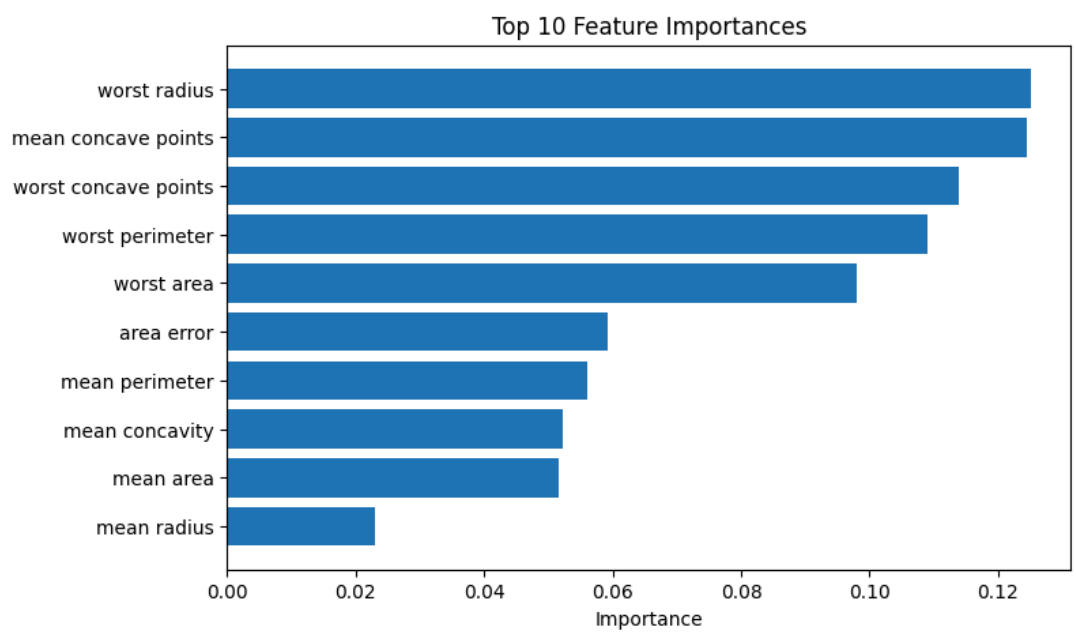
Next steps: Generate code with df New interactive sheet

The top 10 features are displayed based on their importance scores derived from the random forest model. This allows us to understand which variables have the greatest influence on the model's predictions. According to the table above, the most important features are related to concavity, perimeter, and area. These variables capture geometric properties of cell nuclei and play a key role in distinguishing between benign and malignant cases.

```
import matplotlib.pyplot as plt

df_sorted = df.sort_values(by="importance", ascending=False).head(10)

plt.figure(figsize=(8,5))
plt.barh(df_sorted["feature"], df_sorted["importance"])
plt.gca().invert_yaxis()
plt.title("Top 10 Feature Importances")
plt.xlabel("Importance")
plt.show()
```



This plot visualizes the top 10 most important features, providing an intuitive understanding of which variables drive the model's predictions.

```
top_features = df.head(10)["feature"]

X_train_df = pd.DataFrame(X_train, columns=data.feature_names)
X_test_df = pd.DataFrame(X_test, columns=data.feature_names)

X_train_top = X_train_df[top_features]
X_test_top = X_test_df[top_features]

model2 = RandomForestClassifier()
model2.fit(X_train_top, y_train)

print("Reduced model accuracy:", model2.score(X_test_top, y_test))
```

```
Reduced model accuracy: 0.9298245614035088
```

The top 10 most important features are selected and used to train a new random forest model. The performance of this reduced model is then evaluated to determine whether a smaller subset of features can maintain similar predictive accuracy while improving interpretability.

This result highlights the effectiveness of feature selection in reducing model complexity while preserving predictive power. It also suggests that not all features are equally important, reinforcing the idea that identifying key variables is essential when exploring potential causal relationships.

Although feature importance highlights strong associations, it does not necessarily imply causality. Features that significantly impact model performance when removed or altered may be considered more relevant, but further analysis is required to establish true causal relationships.

Overall, this analysis demonstrates that combining machine learning with feature selection techniques can improve model interpretability without sacrificing performance. It also highlights the important distinction between correlation and potential causal relevance in predictive modeling.

Conclusion on accuracy results for RF models:

- RF with full set of predictors: 0.94
- RF with top 10 predictors: 0.94

Overall, this finding reinforces the importance of identifying key predictors, enabling simpler and more interpretable models without compromising performance.

Model 2:XGBOOST

```
!pip install xgboost
```

```
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.7)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)
```

```
from xgboost import XGBClassifier

xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
xgb_model.fit(X_train, y_train)

y_pred_xgb = xgb_model.predict(X_test)

print("XGBoost Accuracy:", accuracy_score(y_test, y_pred_xgb))
```

```
XGBoost Accuracy: 0.9824561403508771
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [07:41:39] WARNING: /__w/xgboost/xgboost/src/le
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

An XGBoost model was trained to further improve predictive performance by leveraging gradient boosting. This method builds models sequentially, focusing on correcting errors from previous iterations.

The XGBoost model achieved an accuracy of approximately 96%, demonstrating strong predictive performance. It performs slightly below the random forest model but still captures complex patterns effectively.

XGBoost provides a powerful alternative to random forest by using boosting instead of bagging. It often achieves high predictive performance by focusing on difficult-to-classify observations. Comparing XGBoost with random forest and logistic regression helps evaluate the trade-off between model complexity and performance.

```
from xgboost import XGBClassifier

# model
xgb_model_top = XGBClassifier(use_label_encoder=False, eval_metric='logloss')

# fit sadece top features ile
xgb_model_top.fit(X_train_top, y_train)

# predict
y_pred_xgb_top = xgb_model_top.predict(X_test_top)

# accuracy
print("XGBoost (Top Features) Accuracy:", accuracy_score(y_test, y_pred_xgb_top))
```

Conclusion on accuracy results for XGBoost models:

- XGBoost with full set of predictors: 0.96
- XGBoost with top 10 predictors: 0.92

The XGBoost model using the reduced set of features achieved an accuracy of approximately 92%, which is slightly lower than the full-feature model. This indicates that while the selected top features capture a significant portion of the predictive information, some additional features still contribute to model performance.

Model 3: Logistic Regression

```
from sklearn.linear_model import LogisticRegression

# model
log_model_full = LogisticRegression(max_iter=10000)

# fit
log_model_full.fit(X_train, y_train)

# predict
y_pred_log_full = log_model_full.predict(X_test)

# accuracy
print("Logistic Regression (Full) Accuracy:", accuracy_score(y_test, y_pred_log_full))
```

```
# model
log_model_top = LogisticRegression(max_iter=10000)

# fit (top features ile)
log_model_top.fit(X_train_top, y_train)

# predict
y_pred_log_top = log_model_top.predict(X_test_top)

# accuracy
print("Logistic Regression (Top Features) Accuracy:", accuracy_score(y_test, y_pred_log_top))
```

In contrast to more complex models such as XGBoost, logistic regression does not benefit from additional features as much, likely because it cannot capture complex nonlinear interactions.

Overall,

This comparison demonstrates that while the full XGBoost model is a powerful model with 0.96 accuracy, it may require careful feature selection or tuning. In contrast, random forest provides more stable performance with fewer features. Both full and reduced random forest models achieve 0.94 accuracy, suggesting that feature selection is highly effective and that the most important predictors capture the majority of the underlying signal.

This highlights the robustness of random forest and its ability to maintain performance even with a simplified feature set, making it a strong choice when interpretability and efficiency are important considerations.

Part II: Finding cause and effect relationship with Causal ML

```
import pandas as pd
!pip install dowhy

from dowhy import CausalModel

# dataframe oluştur
df_full = pd.DataFrame(X, columns=data.feature_names)
df_full["target"] = y
```

```
Collecting dowhy
  Downloading dowhy-0.14-py3-none-any.whl.metadata (18 kB)
Collecting causal-learn>=0.1.3.0 (from dowhy)
  Downloading causal_learn-0.1.4.5-py3-none-any.whl.metadata (4.6 kB)
Collecting cvxpy<2.0.0,>=1.7.1 (from dowhy)
  Downloading cvxpy-1.8.2-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (9.6 kB)
Requirement already satisfied: cython>=3.0 in /usr/local/lib/python3.12/dist-packages (from dowhy) (3.0.12)
Requirement already satisfied: joblib>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from dowhy) (1.5.3)
Requirement already satisfied: networkx>=3.3 in /usr/local/lib/python3.12/dist-packages (from dowhy) (3.6.1)
Requirement already satisfied: numba>=0.59 in /usr/local/lib/python3.12/dist-packages (from dowhy) (0.60.0)
Requirement already satisfied: numpy>2.0 in /usr/local/lib/python3.12/dist-packages (from dowhy) (2.0.2)
Requirement already satisfied: pandas>1.0 in /usr/local/lib/python3.12/dist-packages (from dowhy) (2.2.2)
Requirement already satisfied: scikit-learn>1.0 in /usr/local/lib/python3.12/dist-packages (from dowhy) (1.6.1)
Collecting scipy<=1.15.3 (from dowhy)
  Downloading scipy-1.15.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (61 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 62.0/62.0 kB 5.3 MB/s eta 0:00:00
Requirement already satisfied: statsmodels>=0.14 in /usr/local/lib/python3.12/dist-packages (from dowhy) (0.14.6)
Requirement already satisfied: sympy>=1.10.1 in /usr/local/lib/python3.12/dist-packages (from dowhy) (1.14.0)
Requirement already satisfied: tqdm>=4.64.0 in /usr/local/lib/python3.12/dist-packages (from dowhy) (4.67.3)
Requirement already satisfied: graphviz in /usr/local/lib/python3.12/dist-packages (from causal-learn>=0.1.3.0->dowhy) (0.21
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from causal-learn>=0.1.3.0->dowhy) (3.
Requirement already satisfied: pydot in /usr/local/lib/python3.12/dist-packages (from causal-learn>=0.1.3.0->dowhy) (4.0.1)
Collecting momentchi2 (from causal-learn>=0.1.3.0->dowhy)
  Downloading momentchi2-0.1.8-py3-none-any.whl.metadata (6.1 kB)
Requirement already satisfied: osqp>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from cvxpy<2.0.0,>=1.7.1->dowhy) (1.1
Requirement already satisfied: clarabel>=0.5.0 in /usr/local/lib/python3.12/dist-packages (from cvxpy<2.0.0,>=1.7.1->dowhy)
Requirement already satisfied: scs>=3.2.4.post1 in /usr/local/lib/python3.12/dist-packages (from cvxpy<2.0.0,>=1.7.1->dowhy)
Requirement already satisfied: highspy>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from cvxpy<2.0.0,>=1.7.1->dowhy)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.59->dowh
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>1.0->dowhy) (2
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>1.0->dowhy) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>1.0->dowhy) (2025.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>1.0->dowhy
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.12/dist-packages (from statsmodels>=0.14->dowhy) (1.0.
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dist-packages (from statsmodels>=0.14->dowhy) (2
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.10.1->dowhy) (1.
Requirement already satisfied: cffi in /usr/local/lib/python3.12/dist-packages (from clarabel>=0.5.0->cvxpy<2.0.0,>=1.7.1->c
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from osqp>=1.0.0->cvxpy<2.0.0,>=1.7.1->dow
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from osqp>=1.0.0->cvxpy<2.0.0,>=1.7.1-
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>1.0-
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->causal-learn>=0
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->causal-learn>=0.1.3
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->causal-learn>=
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->causal-learn>=
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib->causal-learn>=0.1.3.0-
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->causal-learn>=0
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi->clarabel>=0.5.0->cvxpy<2.0.0
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->osqp>=1.0.0->cvxpy<2
Downloading dowhy-0.14-py3-none-any.whl (403 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 403.1/403.1 kB 22.5 MB/s eta 0:00:00
Downloading causal_learn-0.1.4.5-py3-none-any.whl (204 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 204.3/204.3 kB 18.4 MB/s eta 0:00:00
Downloading cvxpy-1.8.2-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (1.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.4/1.4 MB 58.0 MB/s eta 0:00:00
Downloading scipy-1.15.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (37.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 37.3/37.3 MB 35.2 MB/s eta 0:00:00
Downloading momentchi2-0.1.8-py3-none-any.whl (11 kB)
Installing collected packages: scipy, momentchi2, cvxpy, causal-learn, dowhy
  Attempting uninstall: scipy
    Found existing installation: scipy 1.16.3
    Uninstalling scipy-1.16.3:
      Successfully uninstalled scipy-1.16.3
  Attempting uninstall: cvxpy
    Found existing installation: cvxpy 1.6.7
    Uninstalling cvxpy-1.6.7:
      Successfully uninstalled cvxpy-1.6.7
Successfully installed causal-learn-0.1.4.5 cvxpy-1.8.2 dowhy-0.14 momentchi2-0.1.8 scipy-1.15.3
WARNING: The following packages were previously imported in this runtime:
  [scipy]
You must restart the runtime in order to use newly installed versions.
```

RESTART SESSION

```
# top 10 feature listesi (zaten df'den geliyor)
top_features_list = df["feature"].head(10).tolist()

results = []

for i in range(1, 11):
```

```
    selected_features = top_features_list[:i]

    print(f"\nUsing top {i} features:", selected_features)

    for feature in selected_features:
        model = CausalModel(
            data=df_full[selected_features + ["target"]],
            treatment=feature,
            outcome="target"
        )

        identified_estimand = model.identify_effect()
        estimate = model.estimate_effect(
            identified_estimand,
            method_name="backdoor.linear_regression"
        )

        print(f"{feature} -> Causal Effect: {estimate.value}")

        results.append({
            "num_features": i,
            "feature": feature,
            "effect": estimate.value
        })
```

```
mean area -> Causal Effect: -0.0009749253606869956
mean radius -> Causal Effect: -0.10024659780908429
```

```
results_df = pd.DataFrame(results)
results_df.head(20)
```

	num_features	feature	effect
0	1	worst radius	-0.077741
1	2	worst radius	-0.077741
2	2	mean concave points	-9.685305
3	3	worst radius	-0.077741
4	3	mean concave points	-9.685305
5	3	worst concave points	-5.842190
6	4	worst radius	-0.077741
7	4	mean concave points	-9.685305
8	4	worst concave points	-5.842190
9	4	worst perimeter	-0.011275